HTML & CSS

Introduction to Software Systems

01

HTML

HyperText Markup Language - the skeleton of every web page. It tells the browser what content exists and what it means.

Anatomy of HTML

Tags wrap content to give it meaning

```
<h1 class="title">Hello, World!</h1>
```

Opening tag | Attribute | Content | Closing tag

Full HTML document

```
<!DOCTYPE html>        <- Always start with this!
<html lang="en">
  <head>
    <title>My Page</title>          <- Browser tab title
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>Hello!</h1>                 <- Content goes here
    <p>This is a paragraph.</p>
  </body>
</html>
```

Common HTML Elements

The tags you'll use every day

Text

```
h1 to h6  Headings (h1 = biggest)
p  Paragraph
strong  Bold text
em  Italic text
a href='...'  Link
```

Structure

```
header  Page header
nav  Navigation links
main  Main content
section  A section
footer  Page footer
div  Generic container
```

Media

```
img src='' alt=''  Image
video  Video player
audio  Audio player
```

Forms

```
form  Form wrapper
input type=''  Text field
button  Clickable button
label  Label for input
select  Dropdown
```

Semantic HTML

Use the right tag - it matters for accessibility & SEO, also makes it easier to read

Non-Semantic (bad)

```
<div class="header">
  <div class="nav">...</div>
</div>
<div class="main-content">
  <div class="article-title">
    News Story
  </div>
  <div class="text">...</div>
</div>
<div class="footer">...</div>
```

Semantic (good)

```
<header>
  <nav>...</nav>
</header>
<main>
  <article>
    <h1>News Story</h1>
    <p>...</p>
  </article>
</main>
<footer>...</footer>
```

Semantic tags tell browsers AND screen readers what your content MEANS - not just how it looks.

02

CSS

Cascading Style Sheets - controls color, fonts, layout, and everything you can see. HTML is the skeleton; CSS is the skin.

CSS Basics

Selectors, properties, and values

style.css

```
/* How a CSS rule is written */
selector {
  property: value;
}

/* Examples */
h1 {
  color: navy;
  font-size: 2rem;
}

.card {
  background: white;
  padding: 1rem;
  border-radius: 8px;
}

#hero {
  background: #1A2340;
  color: white;
}
```

THREE WAYS TO SELECT

`h1 { }`

Tag selector

Styles EVERY h1 on the page

`.card { }`

Class selector

Styles anything with class="card"
Most common - reusable!

`#hero { }`

ID selector

Styles the element with id="hero"
Unique per page - use sparingly

The Box Model

Every HTML element is a rectangular box

box-model.css

```
.box {
  width: 200px;    /* content */
  padding: 16px;   /* inside  */
  border: 2px solid navy;
  margin: 24px;    /* outside */
}
```

CSS Layout: Flexbox

Arrange items in a row or column - your most-used layout tool

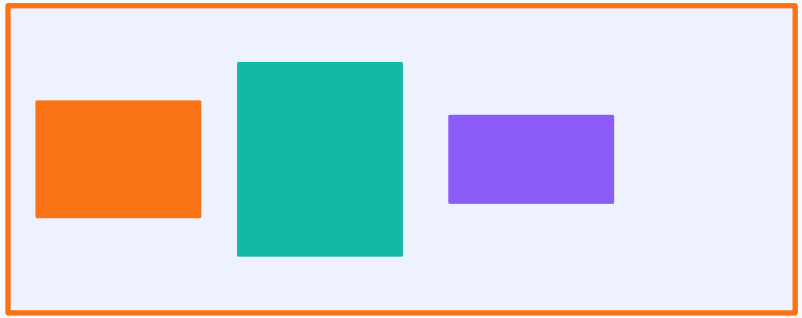
flexbox.css

```
.container {
  display: flex;
  flex-direction: row;        /* or column */
  justify-content: center;    /* main axis  */
  align-items: center;        /* cross axis */
  gap: 1rem;                  /* space between */
}

/* Center ANYTHING */
.center-me {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

justify-content: space-between

align-items: center (cross axis)



Flexbox is perfect for: navbars, button groups, centering things, card rows.

Responsive Design

Making your page look good on every screen size

responsive.css

```
/* Mobile-first: write styles for small screens first */
body {
  font-size: 16px;
  padding: 1rem;
}

.cards {
  display: flex;
  flex-direction: column;  /* stack on mobile */
  gap: 1rem;
}

/* Then add styles for larger screens */
@media (min-width: 768px) {
  .cards {
    flex-direction: row;     /* side by side on tablet+ */
  }
}

@media (min-width: 1024px) {
  body {
    max-width: 960px;
    margin: 0 auto;          /* center on wide screens */
  }
}
```

Mobile-First

Write base styles for small screens. Add @media queries to adjust for bigger screens. Easier to maintain! (Called mobile-first development)

Common Breakpoints

480px - small phones
768px - tablets
1024px - laptops
1280px - desktops

Viewport Meta Tag

Add this to every HTML head:

```
<meta name="viewport"
  content="width=device-width,
  initial-scale=1">
```

03

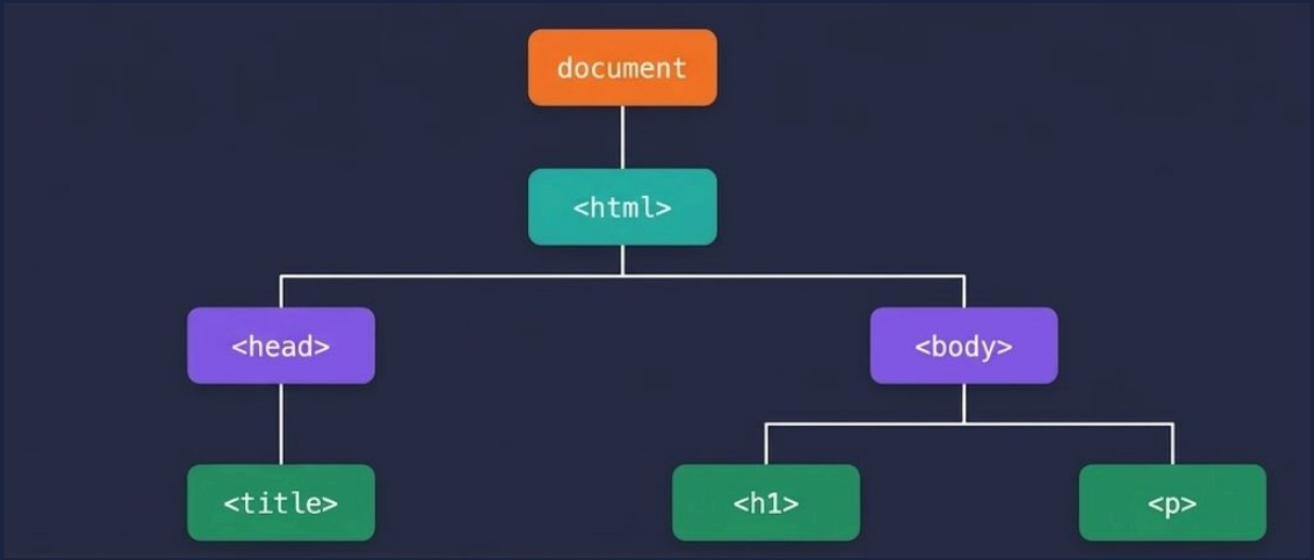
The DOM

The Document Object Model - how your browser turns HTML into a live, programmable tree that JavaScript can read and modify.

What Is the DOM?

Your browser turns HTML into a tree of objects

When a browser loads your HTML, it doesn't just display it - it builds a tree of objects in memory called the DOM. JavaScript can then read and change this tree, instantly updating what users see.



KEY IDEA: The DOM is LIVE - any JavaScript change instantly updates the page.

Manipulating the DOM with JavaScript

Read, change, and react to your page

dom.js

```javascript
// 1. SELECT an element
const title = document.getElementById("main-title");
const btn   = document.querySelector(".search-btn");
const items = document.querySelectorAll("li");  // all <li>

// 2. CHANGE content or style
title.textContent = "New Title";
title.style.color = "orange";
title.classList.add("highlighted");
title.classList.remove("hidden");

// 3. REACT to events (clicks, keypresses...)
btn.addEventListener("click", function() {
  alert("Button clicked!");
});

// Practical example - hide an element
const banner = document.querySelector(".banner");
banner.addEventListener("click", function() {
  banner.style.display = "none";  // hides it
});
```

QUICK REFERENCE

`getElementById()`
Select by id='...'

`querySelector()`
Select by CSS selector

`querySelectorAll()`
Select ALL matches

`textContent`
Change text safely

`style.property`
Change inline CSS

`classList.add/remove`
Toggle CSS classes

`addEventListener()`
Listen for clicks, keys...

04

How Web Pages Are Built

A quick look at different approaches - from plain HTML files to dynamic apps.

Web Development Approaches

Different ways to build web pages

Static

When: Blogs, docs, portfolios

`GitHub Pages, plain HTML`

How: Pre-written HTML/CSS files served directly. What you'll build today!

Server-Side Dynamic

When: Login systems, personalized pages

`PHP, Python/Django, Node.js`

How: Server generates HTML per request using templates and a database

Component-Based (SPA)

When: Dashboards, social apps, complex UIs

`React, Vue, Svelte`

How: JavaScript builds and updates the page in the browser dynamically

Today we are focusing on Static - the foundation everything else is built on.

IN-CLASS ASSIGNMENT

Build a Google Search Page Clone

A search bar that actually searches Google - using only HTML & CSS.

45-60 minutes Solo HTML + CSS only

How the Search Will Work

HTML forms can send data to any URL - including Google!

Google accepts search queries through a simple URL:

```
https://www.google.com/search?q=your+search+here
```

An HTML form pointing to Google with an input named "q" does this automatically!

search.html

```html
<form action="https://www.google.com/search" method="GET">

  <input
    type="text"
    name="q"
    placeholder="Search Google...">

  <button type="submit">Google Search</button>

</form>

<!--
  When user clicks the button, browser navigates to:
  https://www.google.com/search?q=whatever+they+typed

  name="q"  <-- this is the key! Google's search parameter.
  method="GET" <-- puts the value in the URL.
-->
```

What You'll Build

A clean, centered search page - inspired by Google's homepage, or feel free to design it however you want!!

my-search.html

Gmail Images

Google

Search Google...

Google Search | I'm Feeling Lucky

India About Privacy Terms Settings

REQUIREMENTS

Page structure

Centered layout with a logo, search input, and submit button

Resources

Submit the links to any docs/resources that you use

Styled nicely

Custom fonts, colors, shadows, hover effects

Responsive

Looks decent on both mobile and desktop

Tips & Hints

Helpful CSS tricks for the assignment

Center everything vertically

```
body {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}
```

Focus glow on the search bar

```
input:focus {
  outline: none;
  box-shadow: 0 0 0 3px
    rgba(66, 133, 244, 0.4);
}
```

Hover effect on buttons

```
button:hover {
  background: #e8e8e8;
  cursor: pointer;
}
```

Space between nav links

```
nav {
  display: flex;
  justify-content: flex-end;
  gap: 1rem;
  padding: 1rem 2rem;
}
```

Key Takeaways

What to remember from today

1	**HTML = Structure**	HTML defines WHAT is on a page - headings, paragraphs, links, images, forms.
2	**CSS = Appearance**	CSS defines HOW it looks - colors, fonts, layout, animations.
3	**The DOM = Live Tree**	Browsers turn HTML into a tree. JavaScript can change it to make pages interactive.
4	**Semantics Matter**	Use the right tags (header, nav, main, footer) for accessibility and SEO.
5	**Flexbox for Layout**	display: flex is your best friend for centering and arranging elements.
6	**Forms Send Data**	- A form with method GET sends input values as URL params - even to Google!

Open your editor.

The best way to learn HTML & CSS is to write it.
AI can write it but you still need to know how to read your own code.

Good luck with the assignment!